Hugging Face LLM Kursu

Bölüm 5: 🤗 Datasets Kütüphanesi

5.1 – 5.7 · Veriyi yüklemek, temizlemek, ölçeklemek, paylaşmak ve aramak



Sunan: [Ad Soyad]

Kaynak: huggingface.co/learn/llm-course/chapter5 (Apache-2.0) · Türkçe çeviri PDF

Yol haritası

İçindekiler

5.1 Giriş
Neden 🤗 Datasets'e derin dalış?

5.2 Veri setim Hub'da değilse?
Yerel ve uzak dosyalar

5.3 Dilimleme ve parçalama
filter · map · Pandas · split · kaydet

5.4 Büyük veri?
Bellek eşleme ve streaming

5.5 Kendi veri setin
GitHub API → Hub → veri kartı

5.6 FAISS ile anlamsal arama
Embedding + indeks

5.7 🤗 Datasets, tamam!
Özet ve kazanımlar

5.1

Giriş

Yüzeyin altına iniyoruz

Bu bölümün başlıkları

- Bölüm 3'ün 3 adımı
- Bu bölümün yanıtlayacağı sorular

Giriş: yüzeyden derinliğe

Bölüm 3'te gördüğümüz 3 adım



Yüzey: Bölüm 3

Derinlik: Bölüm 5

- yerel/uzak dosyalar
- filter · map
- batched · num_proc
- Pandas ↔ Dataset
- bellek eşleme
- streaming
- Hub'a push
- FAISS

Bu bölümde yanıt arayacağımız sorular

Veri setiniz Hub'da değilse ne yaparsınız?	5.2
Veri setini nasıl dilimler, parçalarsınız? (Ya Pandas gerekirse?)	5.3
Veri seti RAM'inizi eritecek kadar büyükse?	5.4
"Bellek eşleme" ve Apache Arrow nedir?	5.4
Kendi veri setinizi oluşturup Hub'a nasıl gönderirsiniz?	5.5

➜ Bölüm 6 (tokenizasyon) ve Bölüm 7 (ince ayar) için hazırlık

5.2

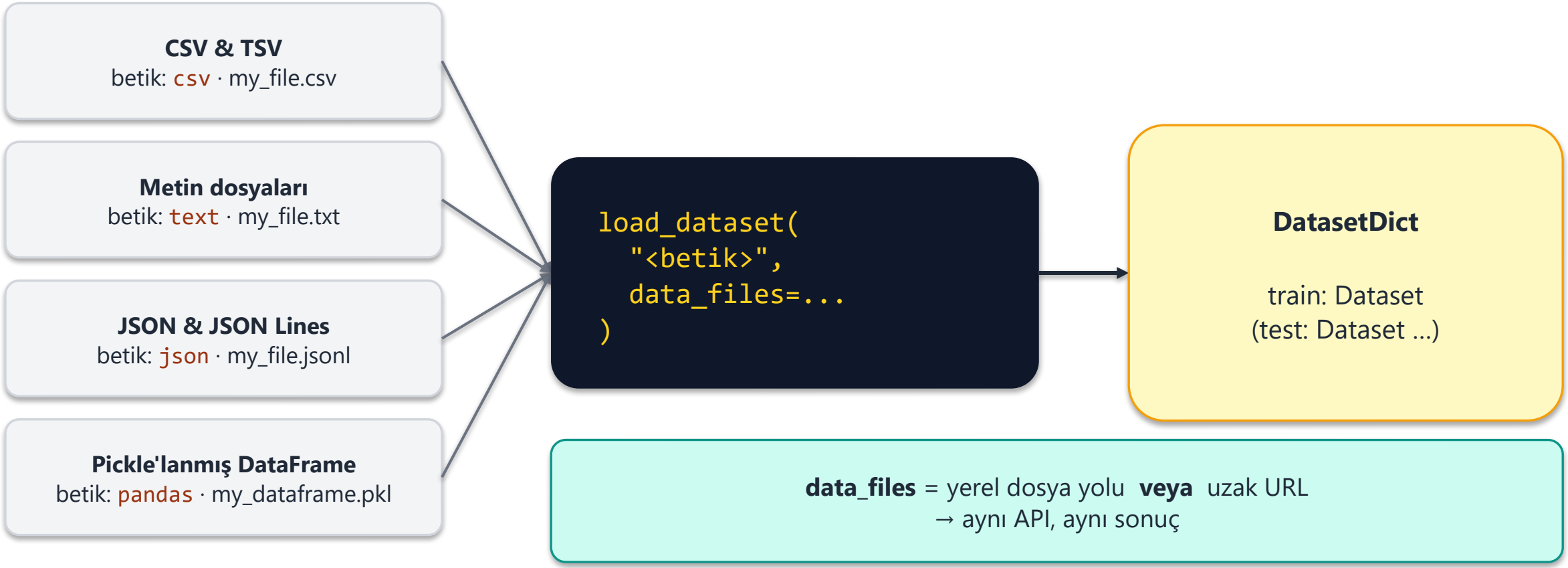
Veri setim Hub'da değilse ne olur?

Yerel diskten ve uzak sunuculardan veri yüklemek

Bu bölümün başlıkları

- Yerel ve uzak veri setleriyle çalışmak
- Yerel bir veri seti yüklemek
- Uzak bir veri seti yüklemek

Yerel ve uzak veri setleriyle çalışmak

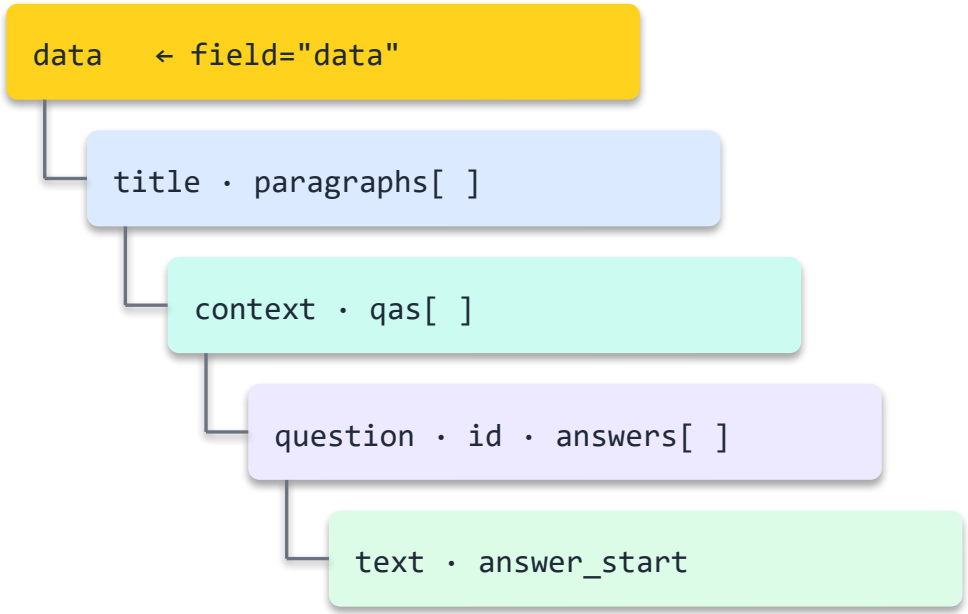


Örnek: `load_dataset("csv", data_files="my_file.csv")` · `load_dataset("json", data_files="my_file.jsonl")`

Yerel bir veri seti yüklemek



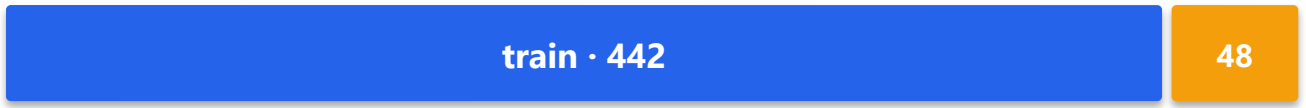
SQuAD-it: iç içe (nested) JSON yapısı



Bölümleri tek DatasetDict'te toplamak

```
data_files = {"train": "SQuAD_it-train.json",
              "test": "SQuAD_it-test.json"}
squad_it_dataset = load_dataset(
    "json", data_files=data_files, field="data")
```

```
DatasetDict · features: ['title', 'paragraphs']
```



Tek dosya verilirse varsayılan: yalnızca `train` bölümü

data_files esnekliği ve otomatik açma

Tek dosya

```
"my_file.json"
```

Dosya listesi

```
["a.json", "b.json"]
```

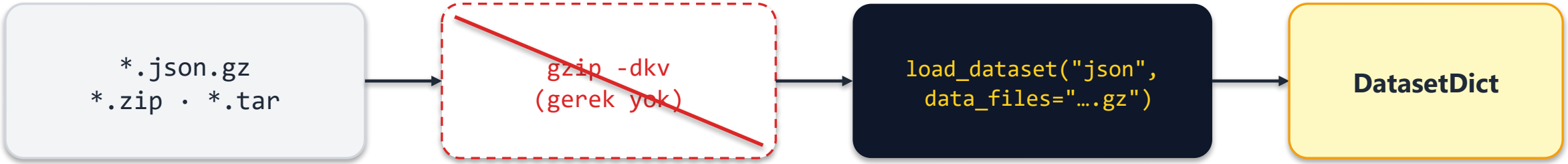
Sözlük: bölüm → dosya

```
{"train": "tr.json",
 "test": "te.json"}
```

Glob deseni

```
"*.json"
```

Otomatik açma: sıkıştırılmış dosyayı doğrudan ver




```
data_files = {"train": "SQuAD_it-train.json.gz", "test": "SQuAD_it-test.json.gz"}
squad_it_dataset = load_dataset("json", data_files=data_files, field="data")
```

Uzak bir veri seti yüklemek




```
url = "https://github.com/crux82/squad-it/raw/master/"
data_files = {
    "train": url + "SQuAD_it-train.json.gz",
    "test": url + "SQuAD_it-test.json.gz",
}
squad_it_dataset = load_dataset(
    "json", data_files=data_files, field="data")
```

Yerel: indir → aç → yükle
(3 adım)

Uzak: tek `load_dataset()` çağrısı
→ aynı DatasetDict

✏ Deneyin: GitHub / UCI ML Repository'den bir veri setini hem yerel hem uzak yükleyin (ekstra: CSV ya da metin)

5.3

Dilimleme ve parçalama zamanı

Gerçek veriyi temizlemek ve dönüştürmek

Bu bölümün başlıkları

- Verimizi dilimlemek ve parçalamak
- Yeni sütunlar oluşturmak
- map() metodunun süper güçleri
- Dataset'lerden DataFrame'lere ve geri
- Doğrulama kümesi oluşturmak
- Bir veri setini kaydetmek

Verimizi dilimlemek ve parçalamak

Unnamed: 0	drugName	condition	review	rating	date	usefulCount
87571	Naproxen	Gout, Acute	"like the previous person mention, I'm a strong believer of aleve..."	9.0	September 2, 2015	36

① Anonim hasta ID'sine benziyor

② Büyük/küçük harf karışık

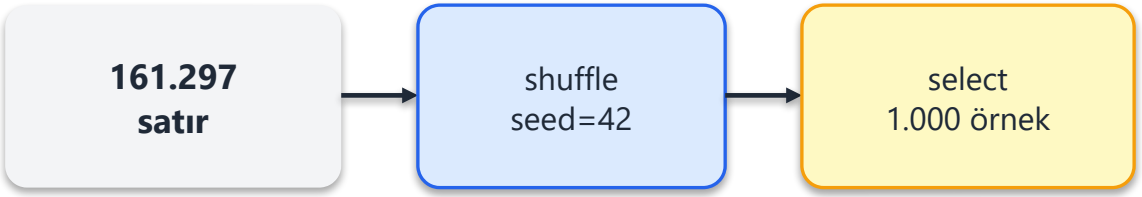
③ \r\n ve ' gibi HTML kodları; uzunluklar çok farklı

TSV = sekmeyle ayrılmış CSV

```
data_files = {"train": "drugsComTrain_raw.tsv",
              "test": "drugsComTest_raw.tsv"}
# \t is the tab character in Python
drug_dataset = load_dataset(
    "csv", data_files=data_files, delimiter="\t")
```

Rastgele örneklem

```
drug_sample = drug_dataset["train"].shuffle(seed=42)
                                   .select(range(1000))
```



Temizleme zinciri: unique → rename → filter → map

1 Hipotezi doğrula

```
unique("Unnamed: 0")
+ assert
```

ID sayısı = satır sayısı ✓

2 Yeniden adlandır

```
rename_column(
"Unnamed: 0",
"patient_id")
```

train 161.297 · test 53.766

3 None'ları at

```
filter(lambda x:
x["condition"]
is not None)
```

AttributeError önlendi

4 Normalleştir

```
map(
lowercase_condition)
```

'adhd', 'birth control' ...

lambda anatomisi

```
lambda <arguments> : <expression>
```

`(lambda x: x * x)(3)` → **9**
`(lambda base, height: 0.5 * base * height)(4, 8)` → **16.0**

Neden önce filter?

`map(lowercase_condition)` →
`AttributeError: 'NoneType' object has no attribute 'lower'`
Çözüm: önce **filter** ile None'ları at, sonra **map**.

Yeni sütunlar oluşturmak

```python
def compute_review_length(example):
    return {"review_length": len(example["review"].split())}

drug_dataset = drug_dataset.map(compute_review_length)
```

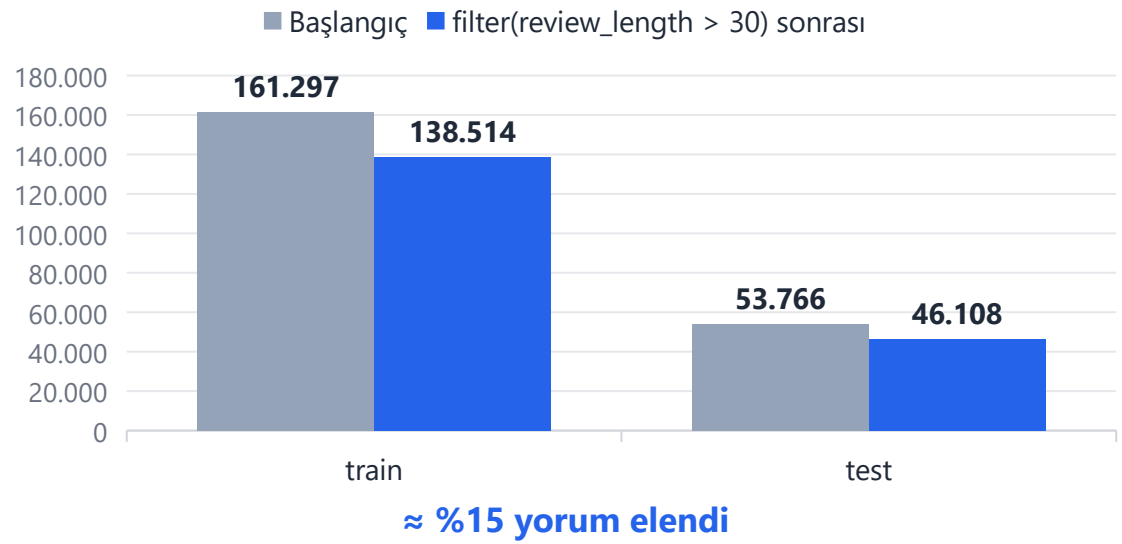
patient_id	review	rating	review_length ★
206461	"It has no side effect, I take..."	9.0	17

Sözlükte yeni anahtar → map yeni sütun ekler

`sort("review_length")` → en kısa yorumlar

"Excellent."	"useless"	"ok"

Her biri 1 kelime: rahatsızlık tahmini için bilgi taşımaz



≈ %15 yorum elendi

HTML karakter kodlarını çözmek

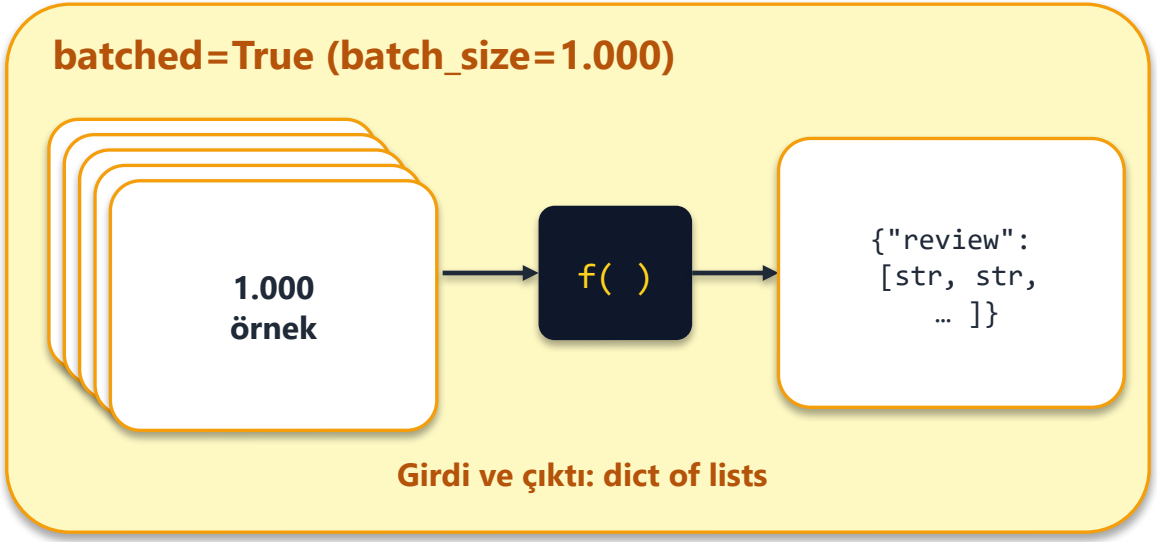
I'm a transformer called BERT	→	I'm a transformer called BERT

```python
drug_dataset.map(lambda x: {"review": html.unescape(x["review"])})
```

🙋 Alternatif: `Dataset.add_column()` — sütunu liste ya da NumPy dizisi olarak ekler

map() metodunun süper güçleri: batched=True





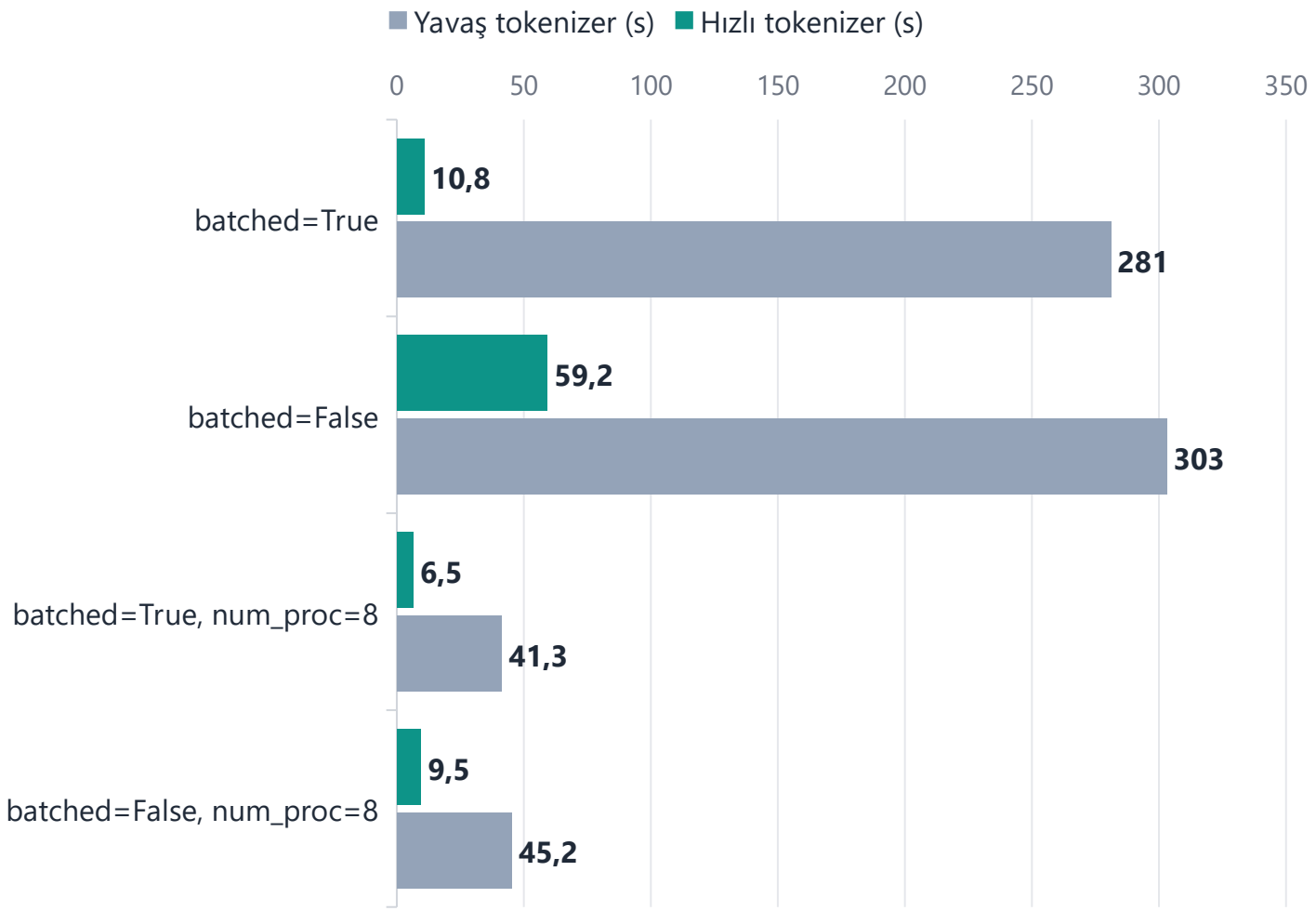

```
new_drug_dataset = drug_dataset.map(
    lambda x: {"review": [html.unescape(o) for o in x["review"]]}, batched=True)
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
def tokenize_function(examples):
    return tokenizer(examples["review"], truncation=True)
```

Neden daha hızlı?

- List comprehension > `for` döngüsü
- Öğelere tek tek değil **topluca** erişim
- Hızlı tokenizer'lar (Rust) batch'i paralel işler

map() hız karşılaştırması: batched ve num_proc



≈30× Hızlı + batched (10,8 s) vs yavaş + batch'siz (5 dk 3 s)

≈6× Hızlı tokenizer'da batch kazancı: 59,2 s → 10,8 s (Rust paralelliği)

num_proc=8 Yavaş tokenizer: 4 dk 41 s → 41,3 s

⚠ Hızlı tokenizer + `batched=True` ile Python çoklu işleme genelde önerilmez. `num_proc`, fonksiyon kendi içinde paralel değilse faydalı.

map() ile örnek sayısını değiştirmek: sorun

Bir yorum → birden fazla özellik (feature)

1 yorum = 177 token

→ `tokenizer`
`max_length=128`
`return_overflowing_tokens=True`

→ **128** | **49**

`[len(inp) for inp in result["input_ids"]]` → [128, 49]

```python
def tokenize_and_split(examples):
    return tokenizer(
        examples["review"],
        truncation=True,
        max_length=128,
        return_overflowing_tokens=True,
    )

tokenized_dataset = drug_dataset.map(
    tokenize_and_split, batched=True)
```

Girdi
1.000 örnek
(eski sütunlar)

→

Çıktı
1.463 özellik
(yeni sütunlar)

```
ArrowInvalid: Column 1 named condition
expected length 1463 but got length 1000
```

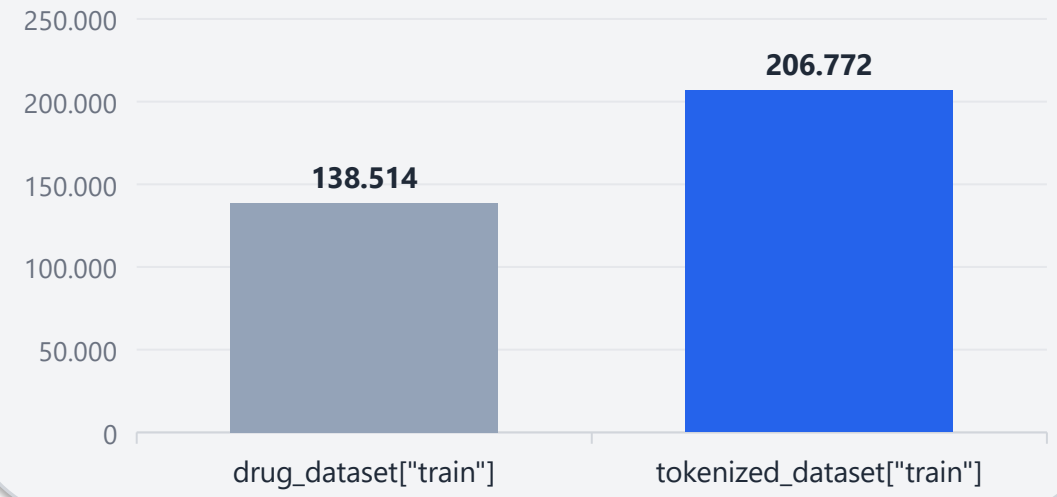
Kural: Bir `Dataset`'te tüm sütunlar aynı uzunlukta olmalı. Çözüm ya eski sütunları **kaldırmak**, ya da onları yeni boyuta **genişletmek** → sonraki slayt

map() ile örnek sayısını değiştirmek: iki çözüm

A) Eski sütunları kaldır

```
tokenized_dataset = drug_dataset.map(
    tokenize_and_split, batched=True,
    remove_columns=drug_dataset["train"].column_names)
```

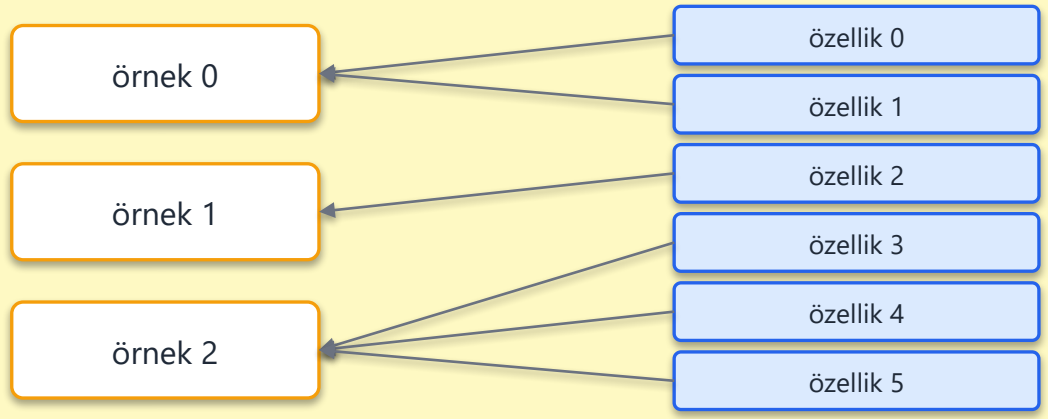
~~condition~~ ~~review~~ ~~rating~~ ~~...~~



B) Eski sütunları genişlet

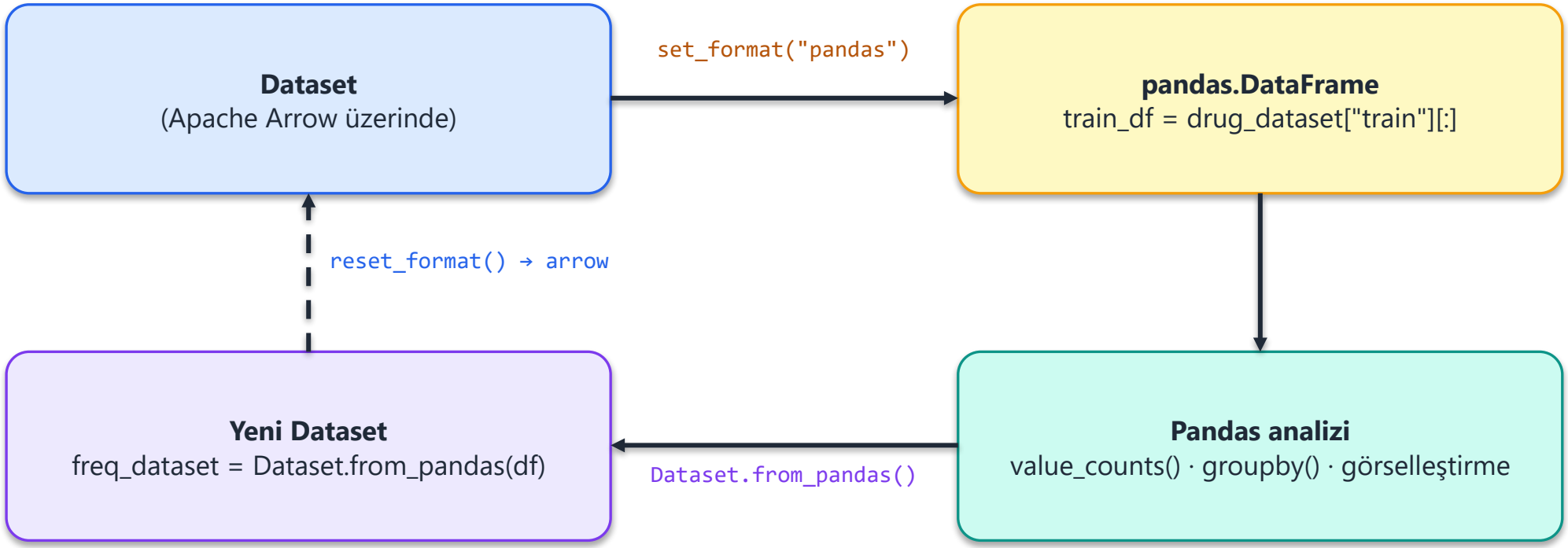
```
result = tokenizer(examples["review"], truncation=True,
    max_length=128, return_overflowing_tokens=True)
sample_map = result.pop("overflow_to_sample_mapping")
for key, values in examples.items():
    result[key] = [values[i] for i in sample_map]
return result
```

`overflow_to_sample_mapping = [0, 0, 1, 2, 2, 2]`



Sonuç: 206.772 / 68.876 satır, eski alanlar korunur

Dataset'lerden DataFrame'lere ve geri



Çıktı formatları

- `arrow`
- `pandas`
- `numpy`
- `torch`
- `tensorflow`
- `jax`

🚨 `set_format()` yalnızca `__getitem__()` dönüş formatını değiştirir. `drug_dataset["train"]` hâlâ bir **Dataset**'tir → DataFrame için tamamını dilimle: `[:]`. Temel veri formatı (Arrow) değişmez; işlem yerinde (in place) yapılır.

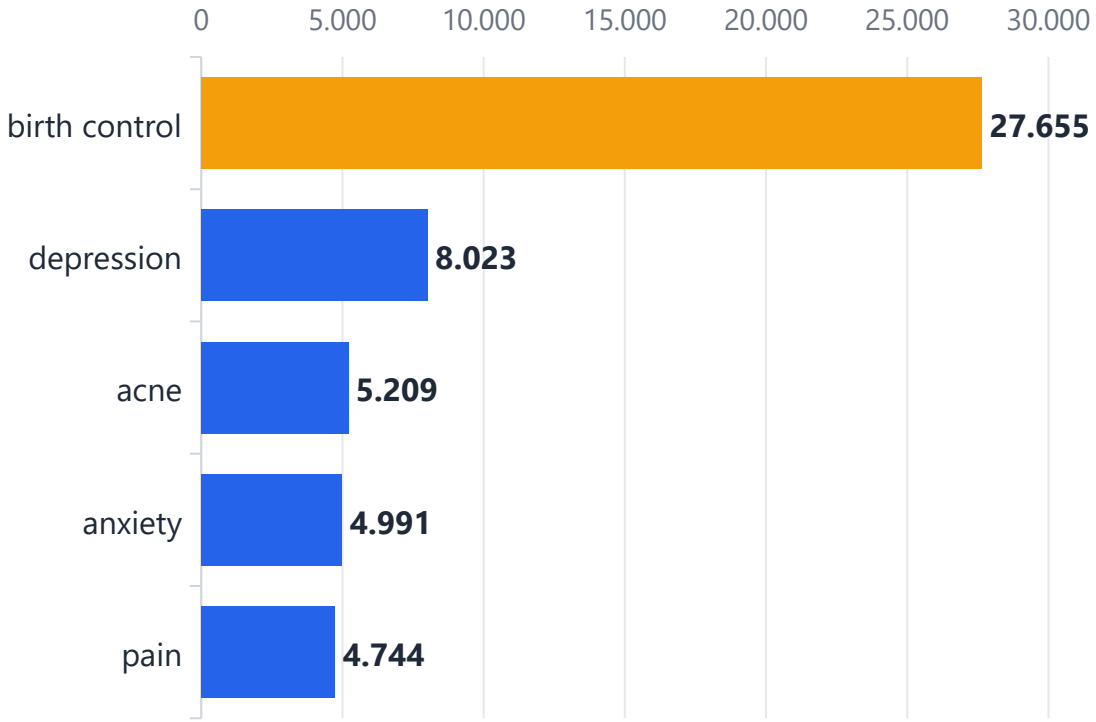
Pandas ile analiz: condition dağılımı

```
drug_dataset.set_format("pandas")
train_df = drug_dataset["train"][:]

frequencies = (
    train_df["condition"]
    .value_counts()
    .to_frame()
    .reset_index()
    .rename(columns={"index": "condition",
                     "count": "frequency"})
)
```

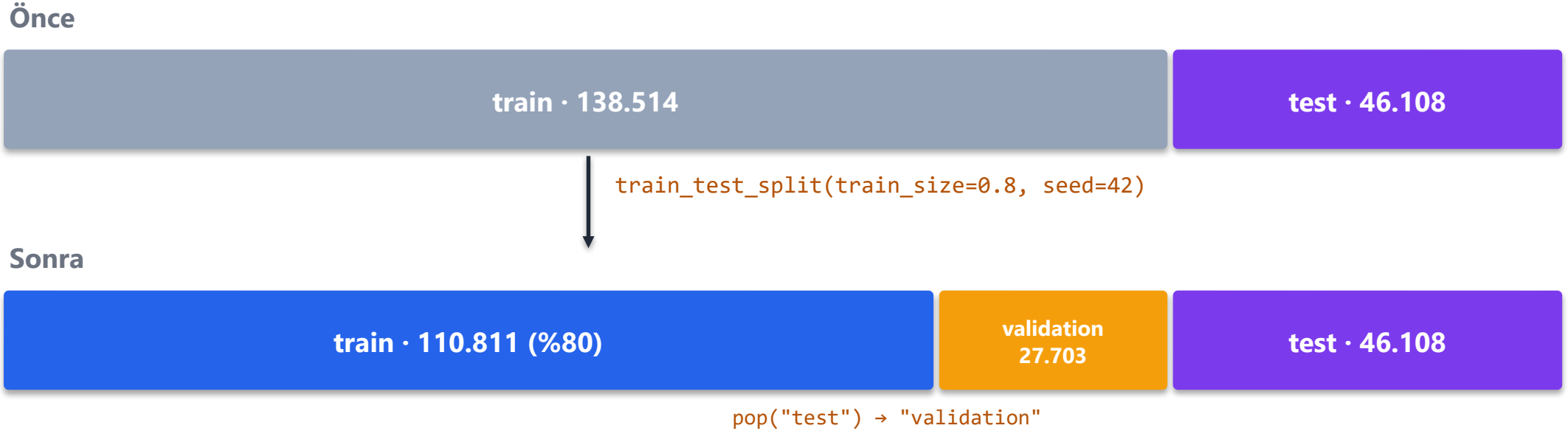
```
from datasets import Dataset

freq_dataset = Dataset.from_pandas(frequencies)
freq_dataset
# Dataset({features: ['condition', 'frequency'],
#          num_rows: 819})
```



✏ Deneyin: İlaç başına ortalama puanı hesaplayın ve sonucu yeni bir `Dataset` içinde saklayın.

Doğrulama kümesi oluşturmak




```
drug_dataset.reset_format()
drug_dataset_clean = drug_dataset["train"].train_test_split(
    train_size=0.8, seed=42)
# Rename the default "test" split to "validation"
drug_dataset_clean["validation"] = drug_dataset_clean.pop("test")
# Add the "test" set to our `DatasetDict`
drug_dataset_clean["test"] = drug_dataset["test"]
```

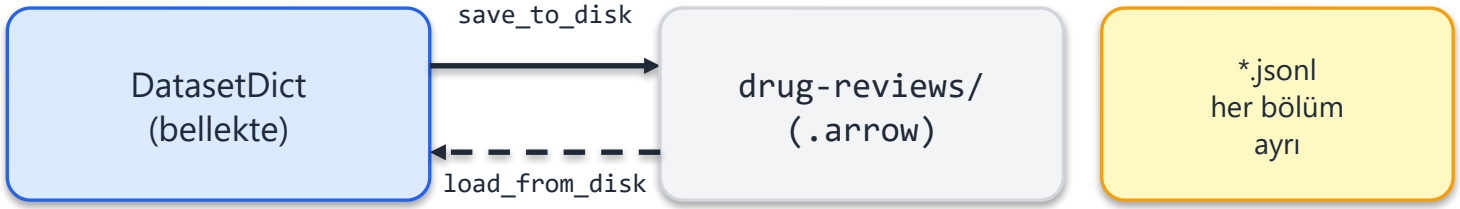


Amaç: test kümesine aşırı uyumu (overfitting) önlemek

Bir veri setini kaydetmek

Veri formatı	Fonksiyon
Arrow	`Dataset.save_to_disk()`
CSV	`Dataset.to_csv()`
JSON	`Dataset.to_json()`

```
save_to_disk("drug-reviews")
drug-reviews/
├── dataset_dict.json
├── test
│   ├── dataset.arrow
│   ├── dataset_info.json
│   └── state.json
├── train
│   ├── dataset.arrow
│   ├── dataset_info.json
│   ├── indices.arrow
│   └── state.json
└── validation
    └── … (aynı yapı)
```




```python
drug_dataset_reloaded = load_from_disk("drug-reviews")

for split, dataset in drug_dataset_clean.items():
    dataset.to_json(f"drug-reviews-{split}.jsonl")

drug_dataset_reloaded = load_dataset("json", data_files={
    "train": "drug-reviews-train.jsonl", ...})
```

```
{"patient_id":141780,"drugName":"Escitalopram","condition":"depression",…}
```

JSON Lines: her satır = bir JSON nesnesi

💡 Fikir 1: Yorumdan rahatsızlığı tahmin eden sınıflandırıcı (Bölüm 3)

💡 Fikir 2: `summarization` pipeline'ı ile yorum özetleri (Bölüm 1)

5.4

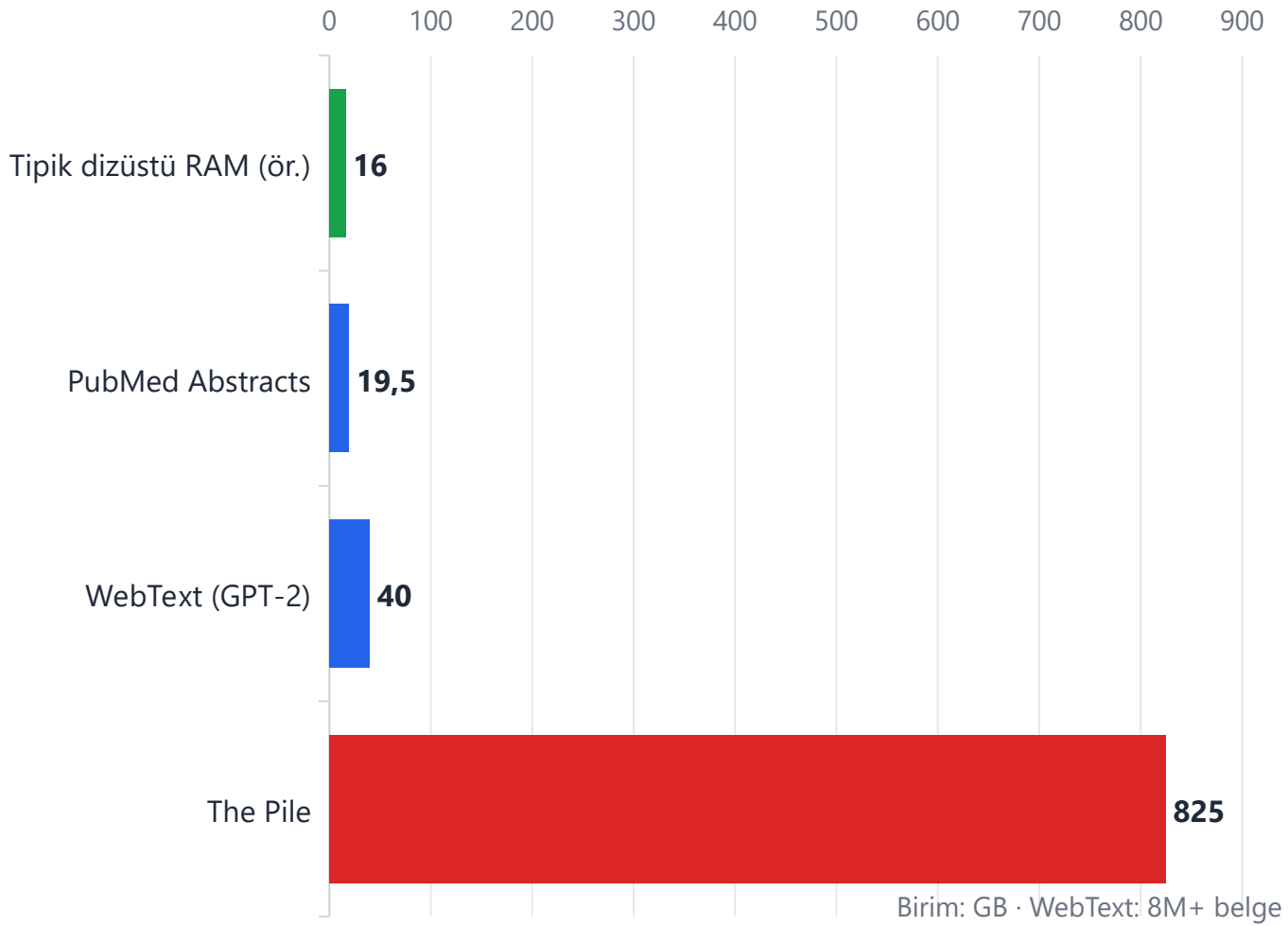
Büyük veri? 🤗 Datasets imdada!

RAM'i ve diski aşan veri setleriyle çalışmak

Bu bölümün başlıkları

- Pile nedir?
- Bellek eşlemenin sihri
- Akışlı (streaming) veri setleri

Problem: veri setleri RAM'den büyük



Bellek (RAM) sorunu

→ Bellek eşleme (memory-mapping)

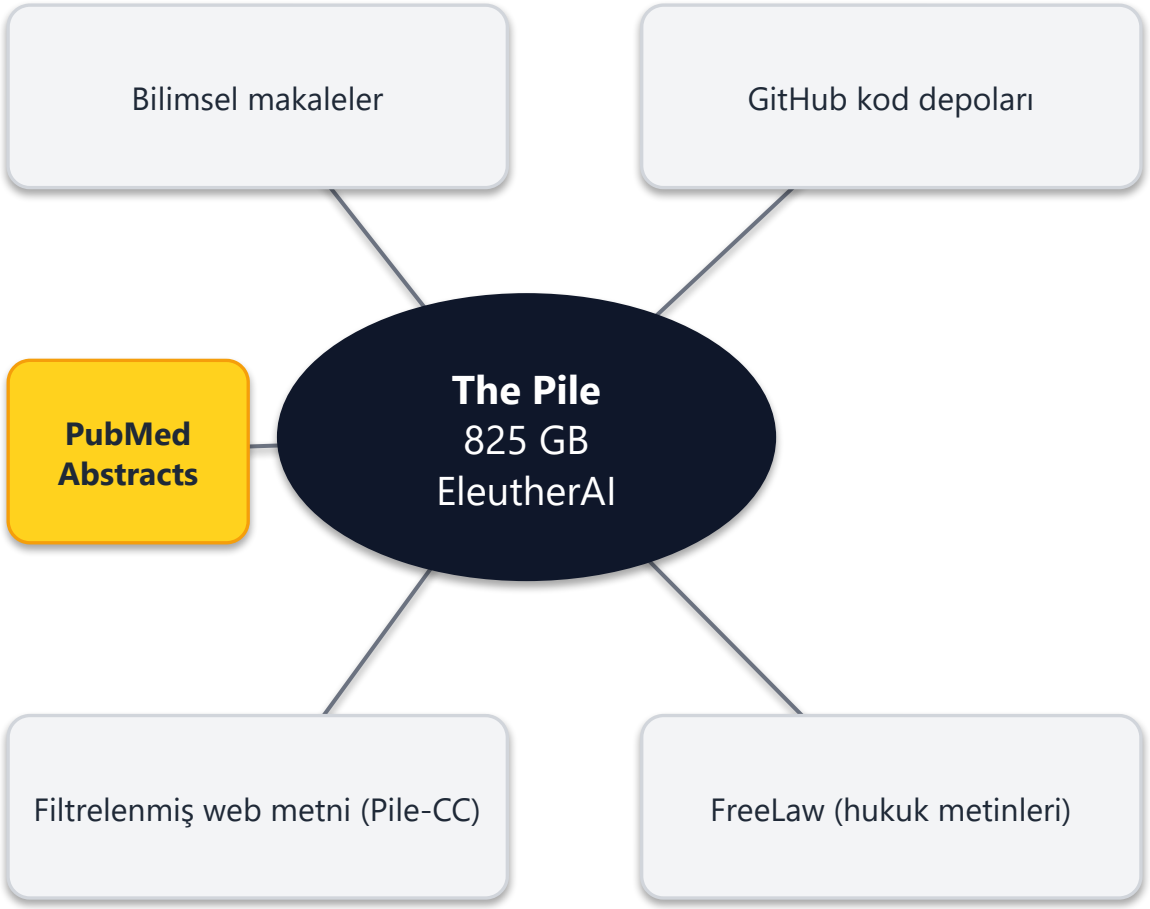
Veri diskte kalır; ihtiyaç duyulan kısım sanal bellek üzerinden okunur. Apache Arrow ile.

Disk sorunu

→ Akış (streaming)

Veri seti indirilmeden, örnekler yineleme sırasında tek tek gelir. `streaming=True`

Pile nedir?



Eğitim derlemi 14 GB'lık parçalar hâlinde · bileşenler ayrı ayrı da indirilebilir

```
!pip install zstandard

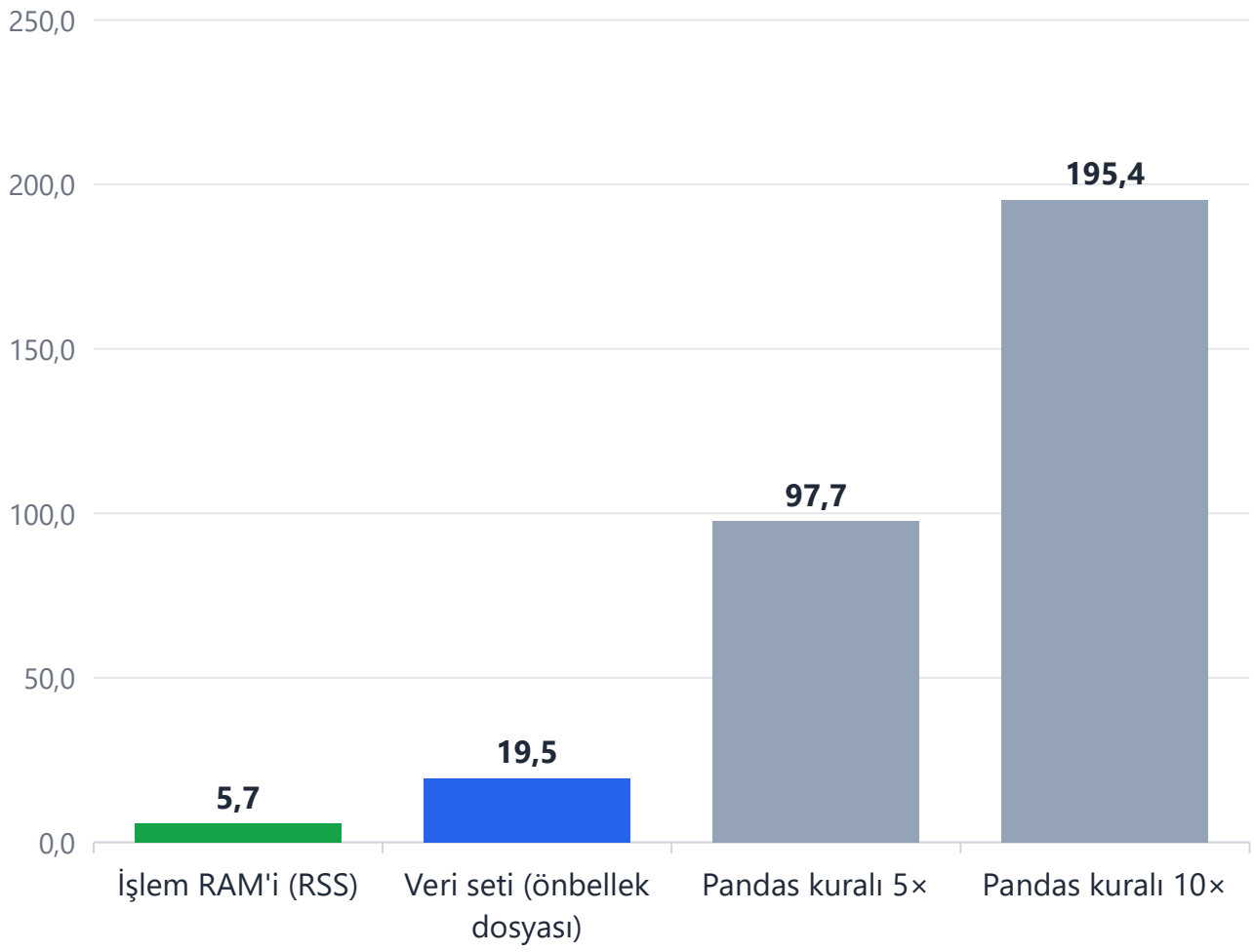
from datasets import load_dataset
# This takes a few minutes to run ...
data_files =
"https://the-eye.eu/public/AI/pile_preliminary_components/PUBMED_title_abstracts_2019_baseline.jsonl.zst"
pubmed_dataset = load_dataset(
    "json", data_files=data_files, split="train")
```

15.518.009
satır

`features:`
`['meta', 'text']`

✎ Disk alanı için:
`download_config=DownloadConfig(delete_extracted=True)`

Bellek eşlemenin sihri

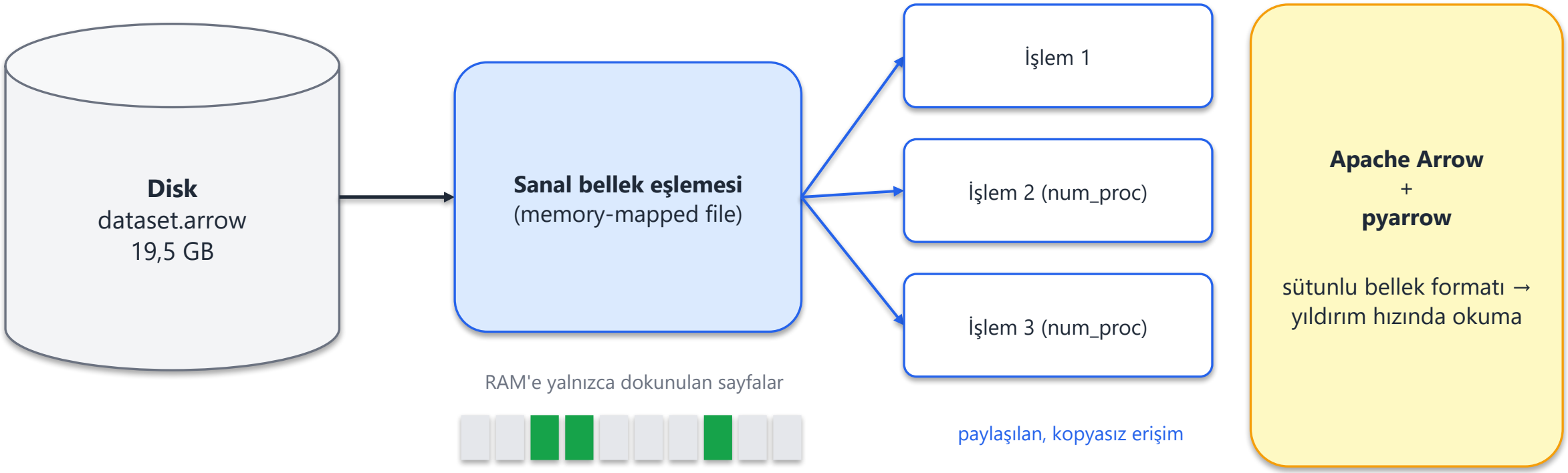



```python
import psutil
rss = psutil.Process().memory_info().rss
print(f"RAM used: {rss / (1024 * 1024):.2f} MB")
# RAM used: 5678.33 MB
```

```python
size_gb = pubmed_dataset.dataset_size / (1024**3)
print(f"Dataset size (cache file) : {size_gb:.2f} GB")
# Dataset size in bytes : 20979437051
# Dataset size (cache file) : 19.54 GB
```

rss = resident set size: işlemin RAM'de kapladığı alan (yorumlayıcı + kütüphaneler dahil).
Pandas'ta genelde veri setinin **5–10 katı** RAM gerekir (Wes McKinney).

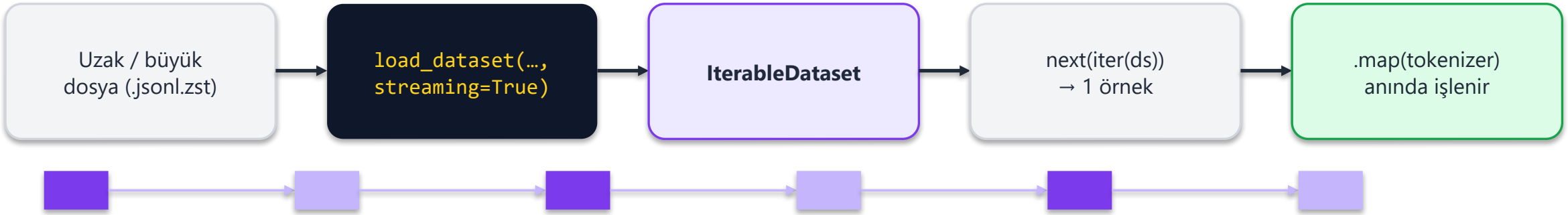
Bellek eşleme nasıl çalışır?




```
code_snippet = """batch_size = 1000
for idx in range(0, len(pubmed_dataset), batch_size):
    _ = pubmed_dataset[idx:idx + batch_size]"""
time = timeit.timeit(stmt=code_snippet, number=1, globals=globals())
```

15.518.009 örnek · 19,5 GB
64,2 s → ≈ 0,304 GB/s

Akışlı (streaming) veri setleri



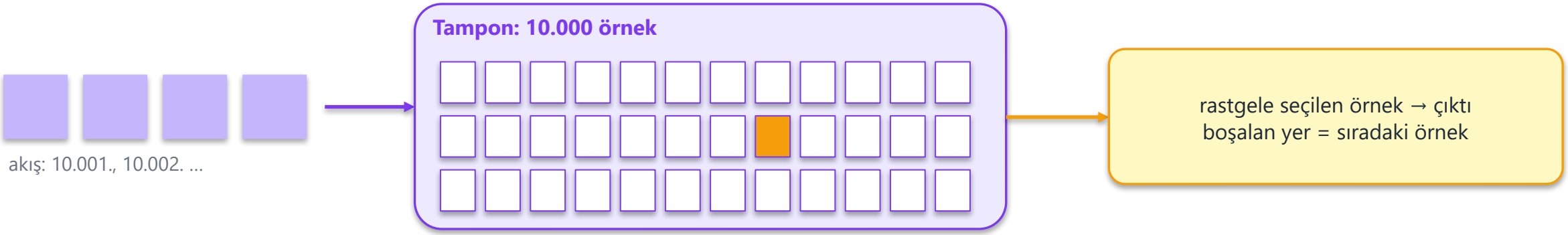

```
pubmed_dataset_streamed = load_dataset(
    "json", data_files=data_files,
    split="train", streaming=True)
next(iter(pubmed_dataset_streamed))

tokenizer = AutoTokenizer.from_pretrained(
    "distilbert-base-uncased")
tokenized_dataset = pubmed_dataset_streamed.map(
    lambda x: tokenizer(x["text"]))
next(iter(tokenized_dataset))
# {'input_ids': [101, 4958, ...], 'attention_mask': [1, 1, ...]}
```

	Dataset	IterableDataset
Erişim	indeks: `ds[i]`	yineleme: `next(iter(ds))`
Depolama	önbellek dosyası (Arrow)	indirmeden akış
map()	hepsine hemen uygulanır	anında (lazy) uygulanır
shuffle()	tüm veri seti	`buffer_size` içinde
Seçim	`select()`	`take()` · `skip()`

Streaming: karıştırma, take() ve skip()

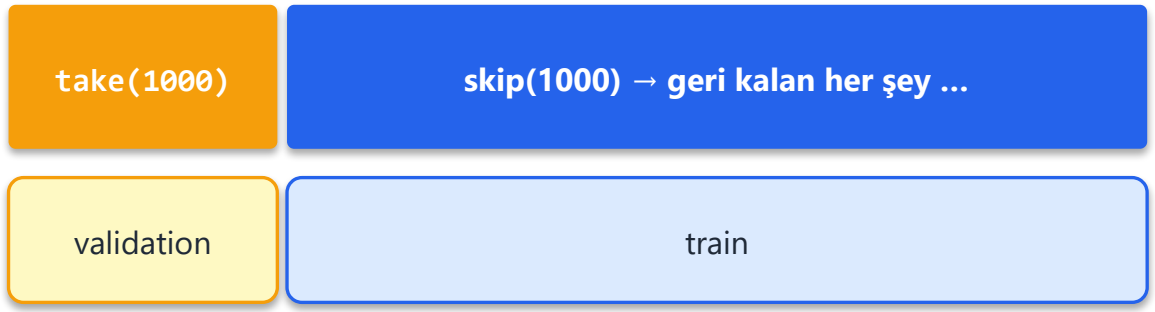
`shuffle(buffer_size=10_000, seed=42)`




```
shuffled_dataset = pubmed_dataset_streamed.shuffle(
    buffer_size=10_000, seed=42)
dataset_head = pubmed_dataset_streamed.take(5)
```

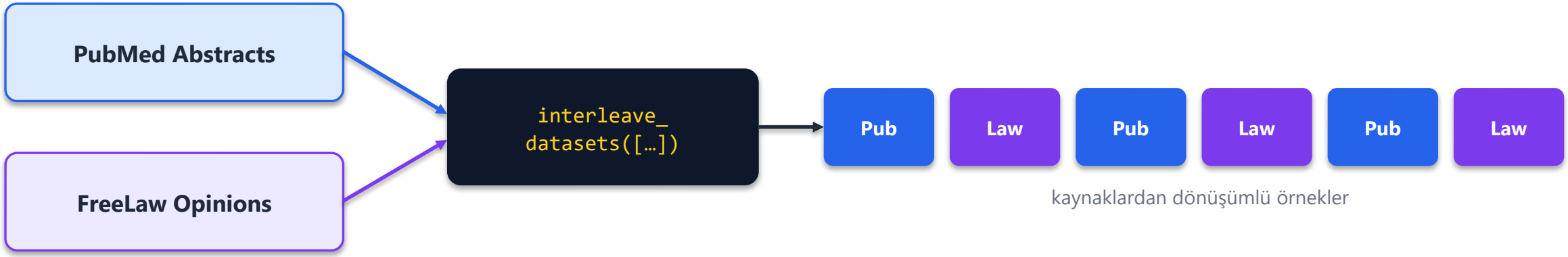
```
# Skip the first 1,000 examples -> training set
train_dataset = shuffled_dataset.skip(1000)
# Take the first 1,000 examples -> validation set
validation_dataset = shuffled_dataset.take(1000)
```

take() / skip() ile bölme



`take()` / `skip()` ≈ akışlı veri setinde `Dataset.select()`

Streaming: interleave_datasets() ve Pile'ın tamamı

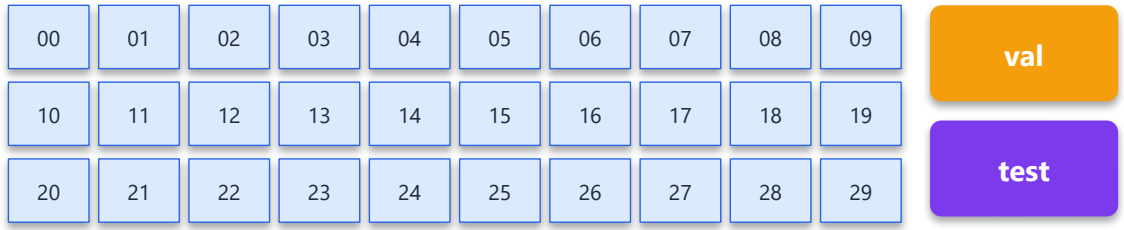



```
law_dataset_streamed = load_dataset("json",
    data_files="https://the-eye.eu/public/AI/pile_preliminary_components/
FreeLaw_Opinions.jsonl.zst",
    split="train", streaming=True)

from itertools import islice
from datasets import interleave_datasets

combined_dataset = interleave_datasets(
    [pubmed_dataset_streamed, law_dataset_streamed])
list(islice(combined_dataset, 2))
```

Pile'ın tamamı (825 GB) akışla



train: 30 parça `{idx:02d}.jsonl.zst` + val + test

✏ Deneyin: `mc4` / `oscar` ile çok dilli akışlı veri seti (ör. İsviçre: DE, FR, IT, Romançça)

5.5

Kendi veri setinizi oluşturmak

GitHub issue'larından Hub'daki bir veri setine

Bu bölümün başlıkları

- Veriyi almak
- Veriyi temizlemek
- Veri setini zenginleştirmek
- Veri setini Hugging Face Hub'a yüklemek
- Bir veri seti kartı oluşturmak

Veriyi almak: GitHub REST API

Kapanma süresi analizi

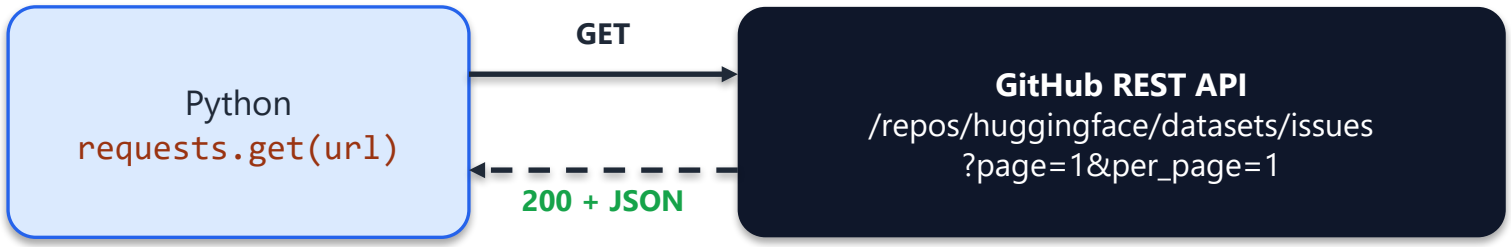
Açık issue / PR'lar ne kadar sürede kapanıyor?

Çok etiketli sınıflandırıcı

bug · enhancement · question

Anlamsal arama motoru

Sorguya en uygun issue'lar → 5.6




```python
import requests

url = "https://api.github.com/repos/huggingface/datasets/issues?page=1&per_page=1"
response = requests.get(url)
response.status_code    # 200
response.json()         # [{'url': ..., 'title': 'Update GooAQ', 'body': ..., ...}]
```

response.json() → alanlar

`title`	`body`
`number`	`user`
`labels`	`state`
`comments`	`created_at`
`closed_at`	`pull_request`

Yazıldığı sırada: **331 açık · 668 kapalı** issue

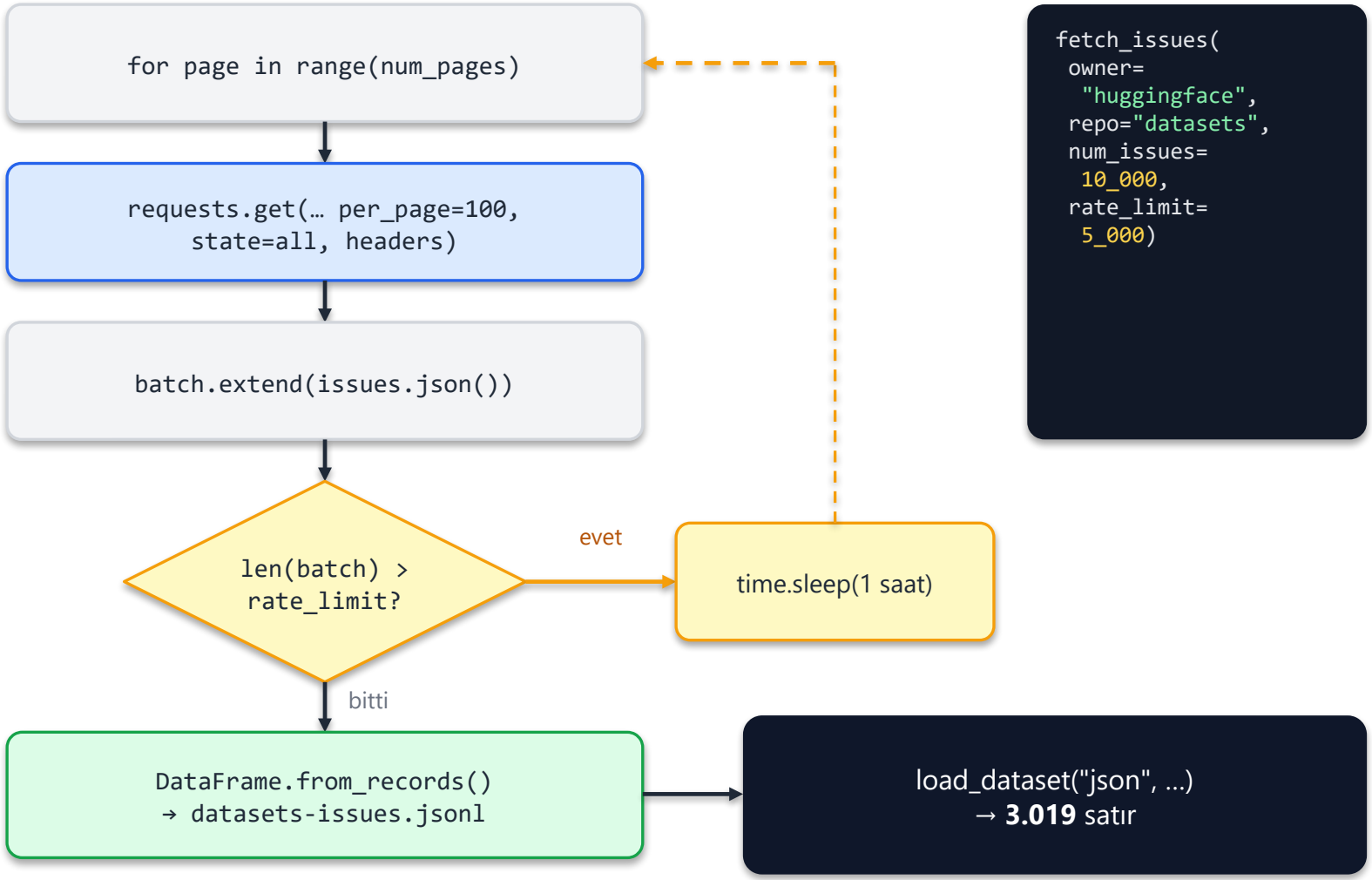
Veriyi almak: istek limiti ve fetch_issues()



GitHub API istek limiti (saatlik)

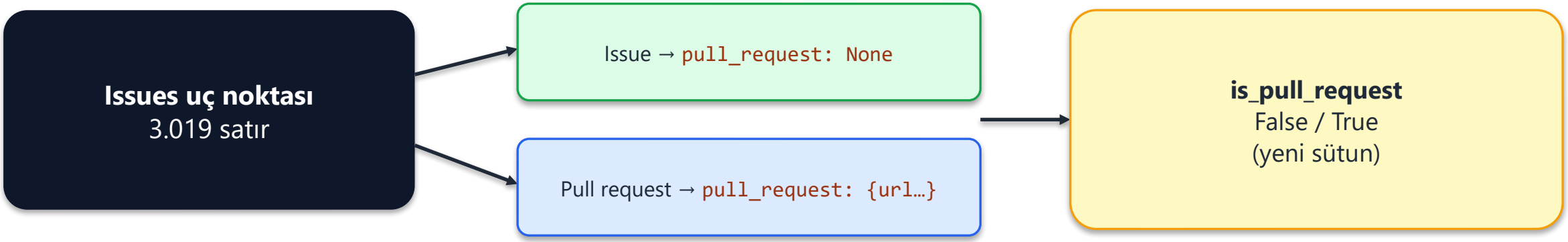
```
headers = {"Authorization":
           f"token {GITHUB_TOKEN}"}
```

⚠ Token'lı notebook'u paylaşma → `.env` + python-dotenv




```
fetch_issues(
 owner=
  "huggingface",
 repo="datasets",
 num_issues=
  10_000,
 rate_limit=
  5_000)
```

Veriyi temizlemek



"GitHub REST API v3 her pull request'i bir issue olarak kabul eder, ancak her issue bir pull request değildir."

```
sample = issues_dataset.shuffle(seed=666).select(range(3))
# >> URL: .../datasets/pull/850   Pull request: {'url': ...}
# >> URL: .../datasets/issues/2773  Pull request: None

issues_dataset = issues_dataset.map(
    lambda x: {"is_pull_request":
               False if x["pull_request"] is None else True})
```

💡 Veri setini olabildiğince **ham** tut → birden fazla uygulamada kullanılabilir

✏ Deneyin: Ortalama kapanma süresi → `filter()` + `set_format("pandas")` + created_at / closed_at

Veri setini zenginleştirmek



her öğenin yalnızca body alanı alınır → yeni liste sütunu

```python
def get_comments(issue_number):
    url = f"https://api.github.com/repos/huggingface/datasets/issues/{issue_number}/
comments"
    response = requests.get(url, headers=headers)
    return [r["body"] for r in response.json()]

# Test our function works as expected
get_comments(2792)

# Depending on your internet connection, this can take a few minutes...
issues_with_comments_dataset = issues_dataset.map(
    lambda x: {"comments": get_comments(x["number"])})
```

title	body	comments ★
Update GooAQ	[GooAQ] dataset was…	["@albertvillanova my tests…"]
…	…	[…, …]

Yorumlar = arama motoru için zengin bilgi kaynağı

Veri setini Hugging Face Hub'a yüklemek




```python
from huggingface_hub import notebook_login
notebook_login()
# terminal: huggingface-cli login

issues_with_comments_dataset.push_to_hub("github-issues")

remote_dataset = load_dataset("lewtun/github-issues", split="train")
remote_dataset
# Dataset({features: [..., 'comments', ..., 'is_pull_request'],
#          num_rows: 2855})
```

2.855 satır
artık toplulukla paylaşılıyor

💡 Alternatif: `huggingface-cli` + biraz Git ile doğrudan terminalden yükleme

Bir veri seti kartı oluşturmak

1. datasets-tagging uygulaması → YAML meta veri etiketleri

2. Veri seti kartı kılavuzu (şablon) → Açıklayıcı bölümler

README.md = veri seti kartı

```
---
task_categories: … · language: en · license: …
---
```

- Veri seti açıklaması
- Veri seti yapısı
- Nasıl oluşturuldu?
- Önyargılar ve riskler

Neden önemli?

- 🎯 Görevle ilgili mi?
- ⚖️ Önyargılar
- Kullanım riskleri
- 🔍 Hub'da bulunabilirlik

✏ Deneyin: Başka bir açık kaynak kütüphanenin issue'larıyla aynı adımları izleyin; ekstra puan: `labels` alanını tahmin eden **çok etiketli sınıflandırıcıyı** ince ayarlayın.

5.6

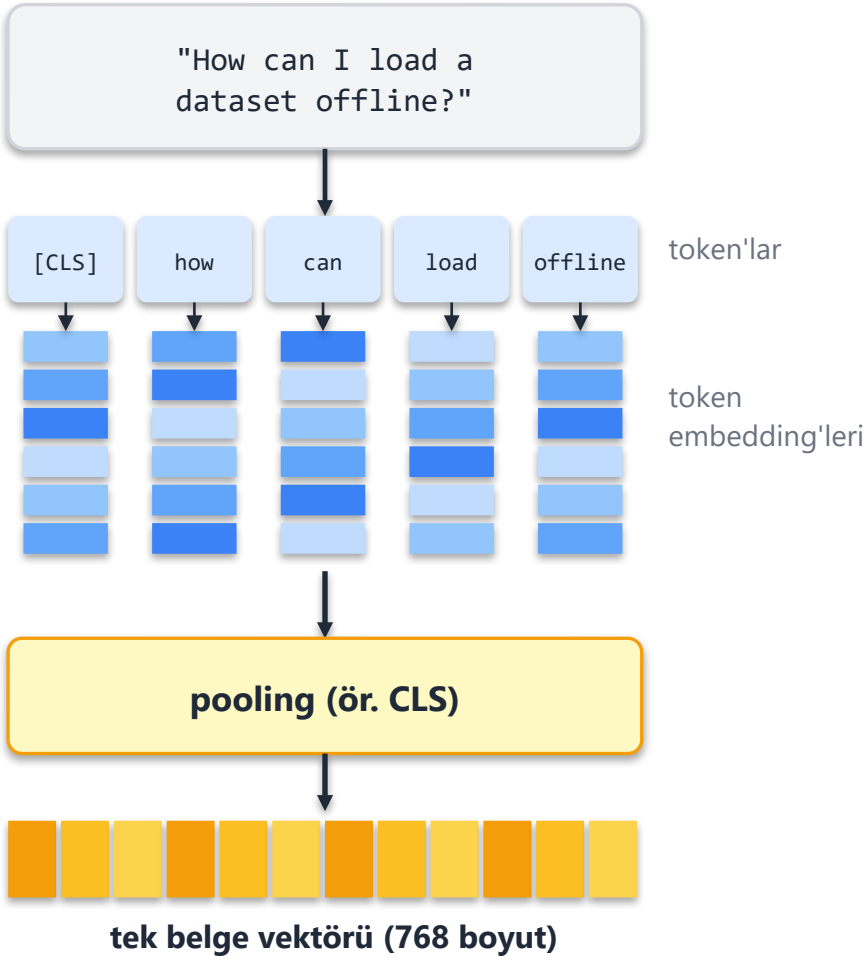
FAISS ile anlamsal arama

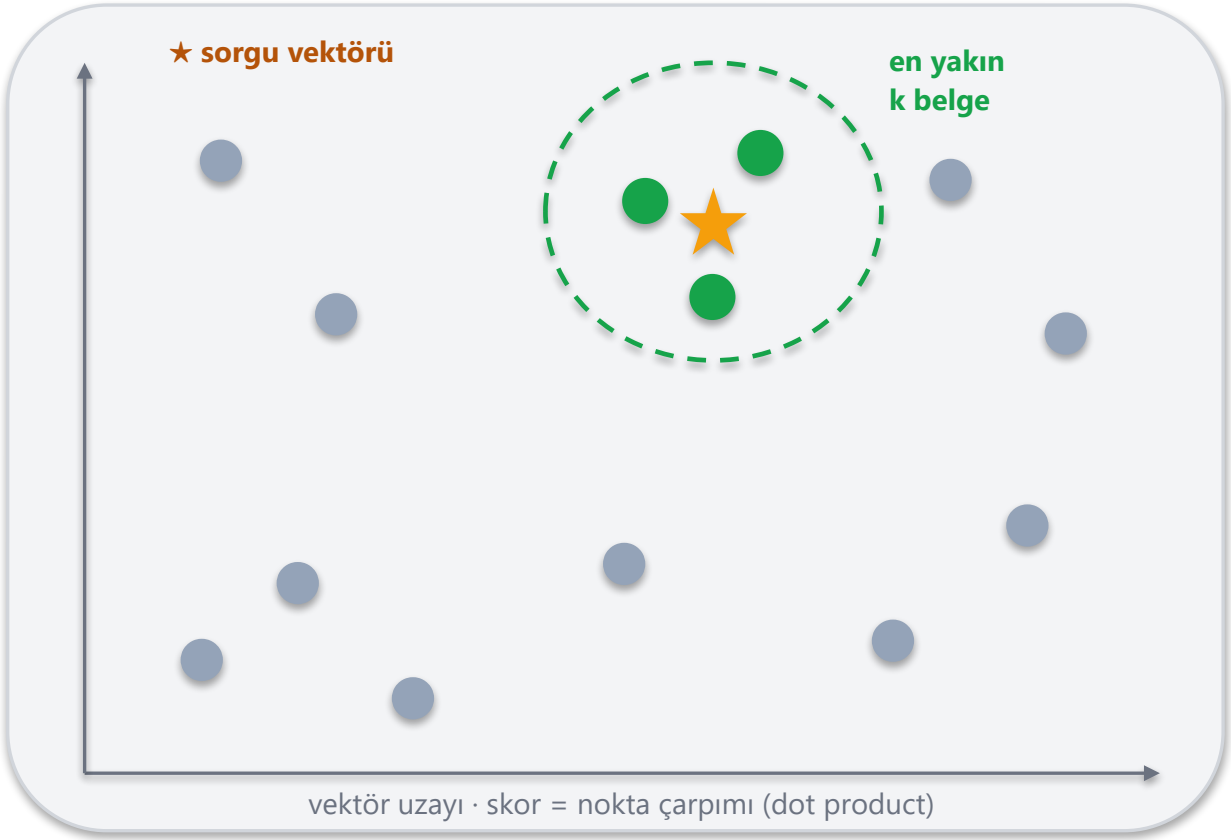
Embedding'lerle anlamına göre arama yapmak

Bu bölümün başlıkları

- Anlamsal arama için embedding'leri kullanmak
- Veri setini yüklemek ve hazırlamak
- Metin embedding'leri oluşturmak
- Verimli benzerlik araması için FAISS kullanmak

Anlamsal arama için embedding'leri kullanmak





Anahtar kelime: birebir kelime eşleşmesi gerekir

Anlamsal: anlamca yakın metinleri bulur

Veri setini yüklemek ve hazırlamak




```python
issues_dataset = issues_dataset.filter(
  lambda x: (x["is_pull_request"] == False
             and len(x["comments"]) > 0))

columns_to_keep = ["title", "body",
                   "html_url", "comments"]
columns_to_remove = set(columns_to_keep)
  .symmetric_difference(columns)
issues_dataset = issues_dataset
  .remove_columns(columns_to_remove)

def concatenate_text(examples):
    return {"text": examples["title"]
        + " \n " + examples["body"]
        + " \n " + examples["comments"]}
```

Hazırlık detayı: comments sütununu patlatmak (explode)

1 satır (issue #2787)	
title	ConnectionError: Couldn't reach …
body	Hello, I am trying to run run_glue.py …
comments	[c1, c2, c3, c4]

explode →

title	body	comments
ConnectionError: …	Hello, I am trying …	c1: the bug code locate in …
ConnectionError: …	Hello, I am trying …	c2: Hi @jinec, From time to time …
ConnectionError: …	Hello, I am trying …	c3: cannot connect, even by …
ConnectionError: …	Hello, I am trying …	c4: I can access … without problem

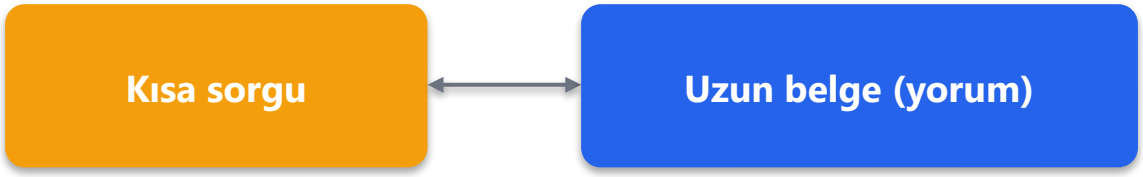
başlık ve gövde çoğaltılır · her satırda tek yorum

```
issues_dataset.set_format("pandas")
df = issues_dataset[:]
comments_df = df.explode("comments", ignore_index=True)

from datasets import Dataset
comments_dataset = Dataset.from_pandas(comments_df)
# Dataset({features: ['html_url', 'title', 'comments', 'body'], num_rows: 2842})
```

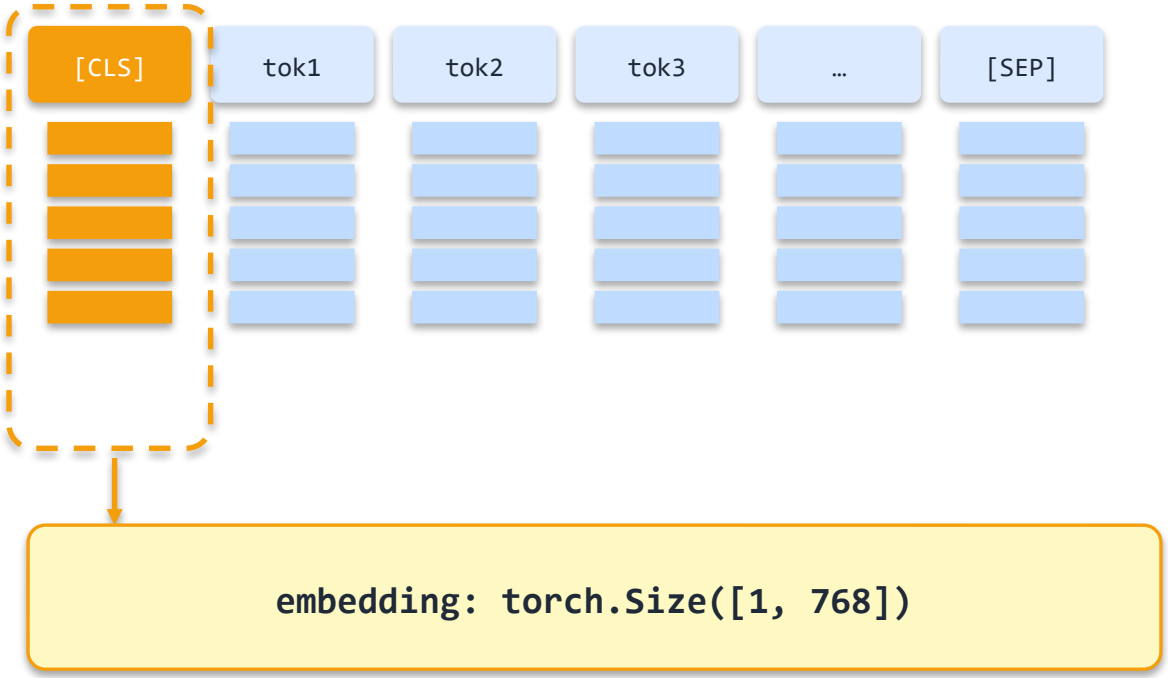
✏ Deneyin: `comments` sütununu Pandas'a başvurmadan, `Dataset.map()` + `batched=True` ile patlatın.
İpucu: batched map'te çıktı satır sayısı girdiden farklı olabilir (dokümantasyon: "Batch mapping").

Metin embedding'leri oluşturmak



Asimetrik anlamsal arama → `multi-qa-mpnet-base-dot-v1`

CLS pooling: last_hidden_state[:, 0]




```python
from transformers import AutoTokenizer, AutoModel

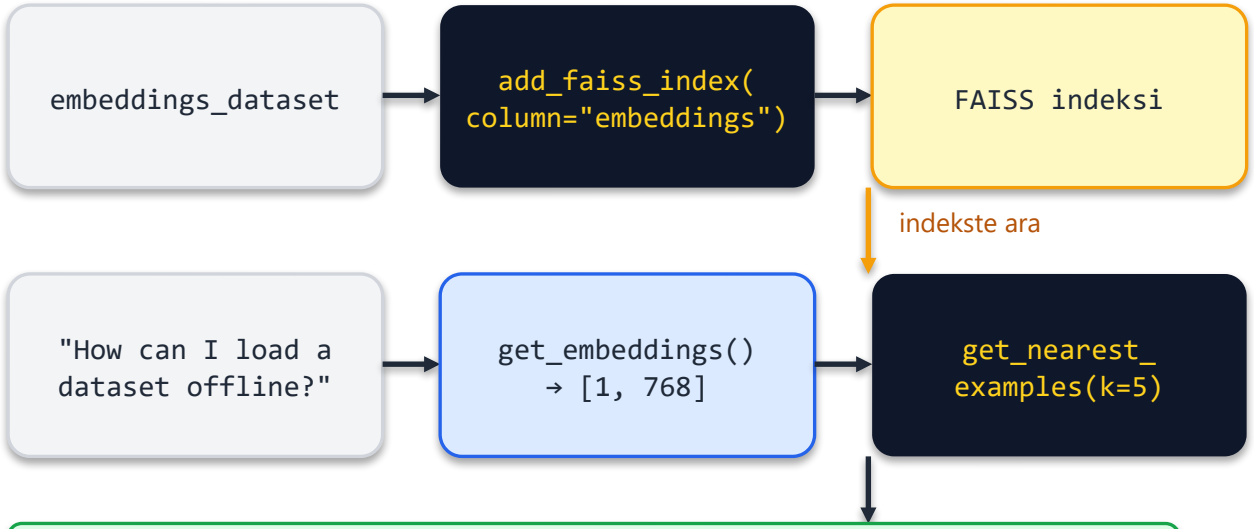
model_ckpt = "sentence-transformers/multi-qa-mpnet-base-dot-v1"
tokenizer = AutoTokenizer.from_pretrained(model_ckpt)
model = AutoModel.from_pretrained(model_ckpt)
device = torch.device("cuda")
model.to(device)

def cls_pooling(model_output):
    return model_output.last_hidden_state[:, 0]

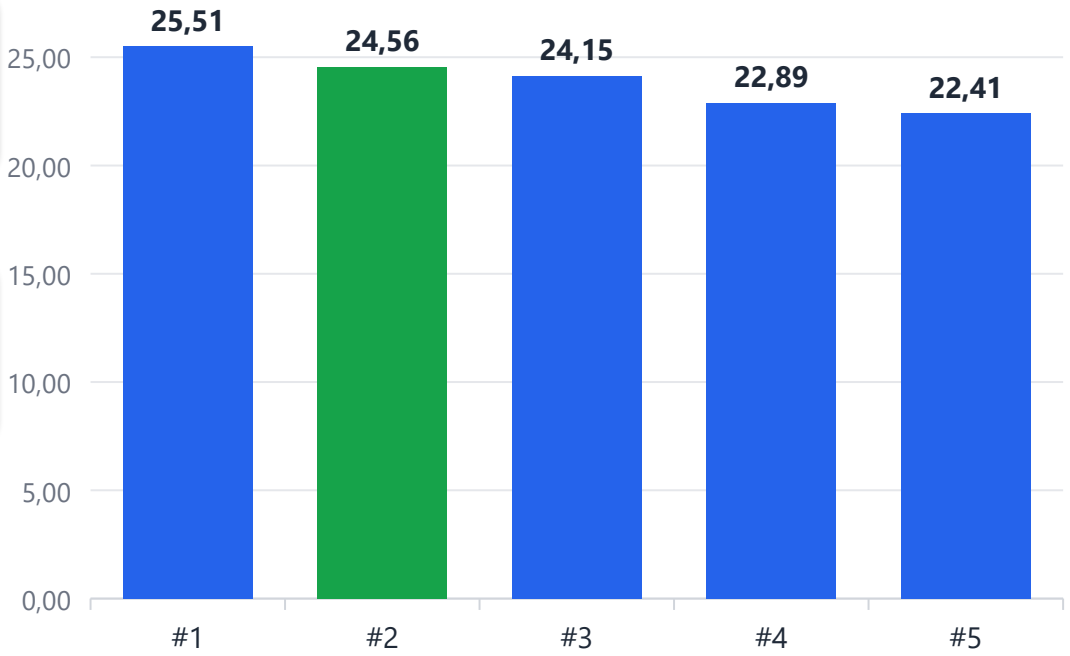
def get_embeddings(text_list):
    encoded_input = tokenizer(text_list, padding=True,
        truncation=True, return_tensors="pt")
    encoded_input = {k: v.to(device)
                     for k, v in encoded_input.items()}
    model_output = model(**encoded_input)
    return cls_pooling(model_output)

embeddings_dataset = comments_dataset.map(
    lambda x: {"embeddings": get_embeddings(x["text"])
               .detach().cpu().numpy()[0]})
```

Verimli benzerlik araması için FAISS kullanmak



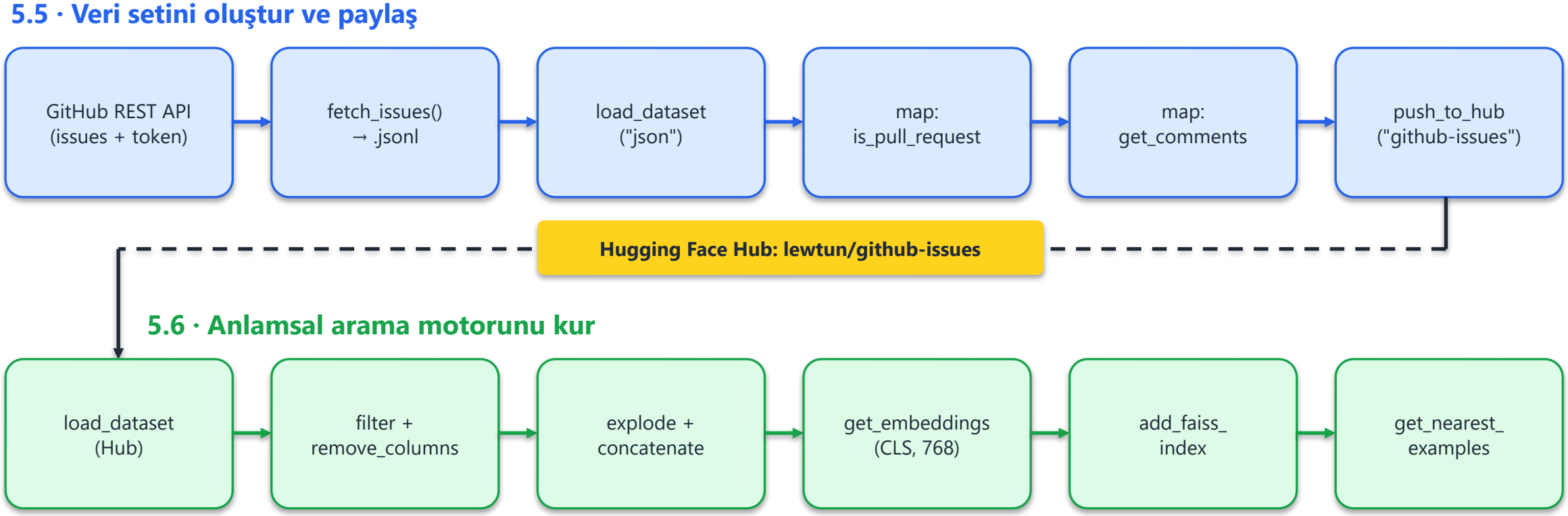

```
embeddings_dataset.add_faiss_index(column="embeddings")
question = "How can I load a dataset offline?"
question_embedding = get_embeddings([question]).cpu().detach().numpy()
scores, samples = embeddings_dataset.get_nearest_examples(
    "embeddings", question_embedding, k=5)
```



Hepsi: #824 “Discussion using datasets in offline mode”

✓ #2: “The local dataset builders (csv, text, json and pandas) are now part of the `datasets` package … You can now use them offline”

Uçtan uca: GitHub issue'larından arama motoruna



Kullanılan araçlar: `load_dataset` · `filter` · `map` · `set_format` · `from_pandas` · `push_to_hub` · `add_faiss_index` · `get_nearest_examples`

5.7

🤗 Datasets, tamam!

Uzun bir turun sonu: kazanımlar

Bu bölümün başlıkları

- Bu bölümde kazandıklarımız
- Bölümün tamamı: uçtan uca harita
- API hızlı başvuru

🤗 Datasets, tamam! Bu bölümde kazandıklarımız

✓ **Veri setlerini her yerden yüklemek**
Hub · dizüstü · uzak sunucu
`load_dataset()` — 5.2

✓ **Veriyi düzenlemek**
map ve filter birlikte
`map()` · `filter()` — 5.3

✓ **Formatlar arasında geçiş**
Pandas · NumPy …
`set_format()` — 5.3

✓ **Kendi veri setini oluşturup paylaşmak**
GitHub API → Hub
`push_to_hub()` — 5.5

✓ **Embedding + FAISS ile anlamsal arama**
Transformer + indeks
`add_faiss_index()` — 5.6

Bonus (5.4)

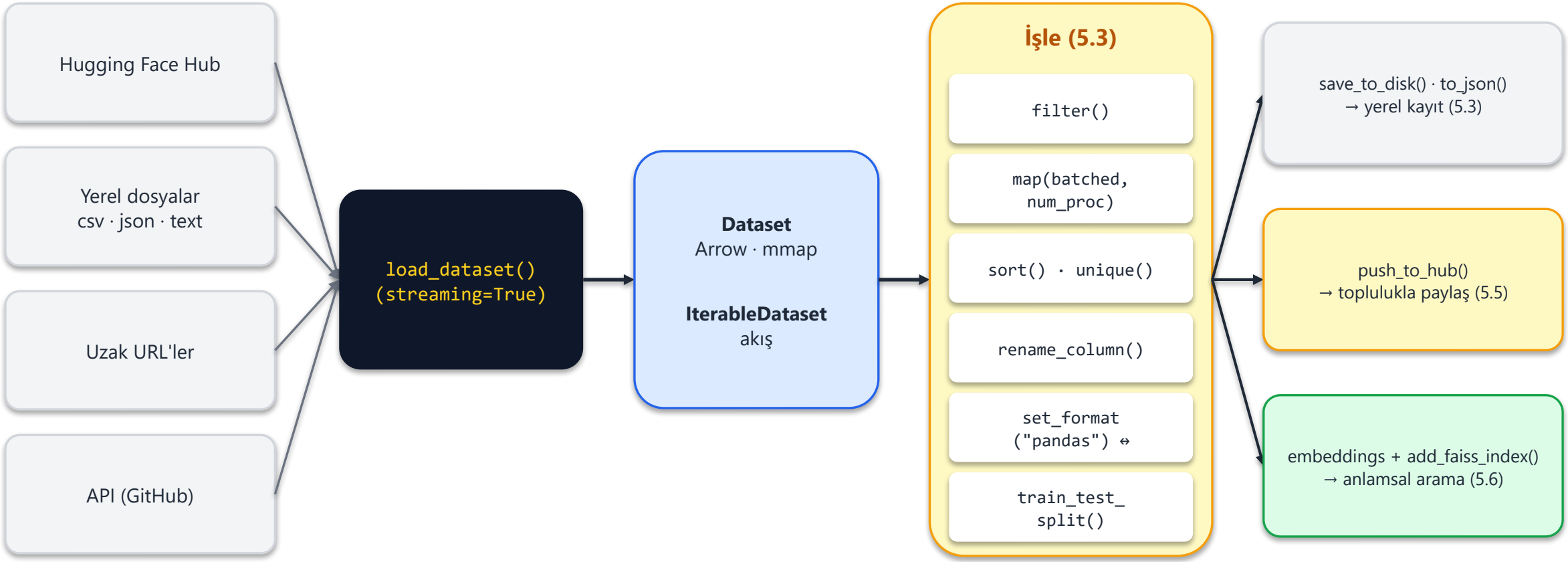
Bellek eşleme ile RAM'den büyük veri

`streaming=True` ile diskten büyük veri

Apache Arrow ile hızlı okuma

➜ Sırada: Bölüm 6 (tokenizer'lar) ve Bölüm 7 (temel NLP görevleri) — önce kısa test!

Bölümün tamamı: uçtan uca harita



Ölçek (5.4): bellek eşleme → RAM sınırı yok · streaming → disk sınırı yok · Apache Arrow → hız

API hızlı başvuru

Fonksiyon	Ne yapar?	Bölüm
`load_dataset(betik, data_files=…)`	Hub / yerel / uzak veri yükler	5.2
`shuffle()` · `select()` · `unique()`	Örneklem alır, benzersiz değerleri bulur	5.3
`rename_column()` · `filter()` · `sort()`	Sütun adlandırır, satır eler, sıralar	5.3
`map(batched=True, num_proc=…)`	Dönüştürür, yeni sütun ekler, satır sayısını değiştirir	5.3
`set_format()` · `reset_format()` · `from_pandas()`	Pandas/NumPy ↔ Dataset geçişi	5.3
`train_test_split()` · `save_to_disk()` · `to_json()`	Bölme ve kaydetme (`load_from_disk`)	5.3
`streaming=True` · `take()` · `skip()` · `interleave_datasets()`	Akışlı büyük veri	5.4
`push_to_hub()`	Veri setini Hub'da paylaşır	5.5
`add_faiss_index()` · `get_nearest_examples()`	FAISS indeksi ve en yakın komşu araması	5.6

Teşekkürler! 🤗

Sorular?

Kaynak: Hugging Face LLM Course — Bölüm 5 (huggingface.co/learn/llm-course/chapter5/1 – /7)
Türkçe çeviri: LLM-Kursu-Bolum5.1-5.7-TR.pdf · Lisans: Apache-2.0 (github.com/huggingface/course)
Veri setleri: SQuAD-it · Drug Review (UCI) · The Pile (PubMed, FreeLaw) · lewtun/github-issues